プログラミングワークショップ 2

青森大学ソフトウェア情報学部
橋本恭能
haship@aomori-u.ac.jp

講義資料について

- この講義で使う資料やプリントは以下のWebページから閲覧できます
- ブラウザを開き、以下のアドレスを入力してください

URL http://haship.github.io

- Webページ上部の【講義資料】をクリック
- 表から【プログラミングワークショップ２】をクリック
- 目次ページをブックマークする

目次

- テーマ：クラスの継承
- 例題：継承の練習
- 課題：壁・反射板・ブロックの共通点

クラスの継承

- オブジェクト指向プログラミングではクラスの継承が重要になる
  - ただし教科書には説明がないので、例題を使って勉強する

例題 犬クラス・猫クラス

- Dogクラス
  - フィールド
    - 年齢 `int age`
  - メソッド
    - 食べる
      `void eat()`
    - 寝る
      `void sleep()`
    - 吠える(犬クラス独自)
      `void bark()`
- Catクラス
  - フィールド
    - 年齢 `int age`
  - メソッド
    - 食べる
      `void eat()`
    - 寝る
      `void sleep()`
    - 鳴く(猫クラス独自)
      `void meow()`

Dogクラス・Catクラス

```
class Dog {

  int age;

  Dog(){
    age = 0;
  }

  void eat(){
    println("Yum!");
  }

  void sleep(){
    println("zzz...");
  }

  void bark(){
    println("WOOF!");
  }

}
```

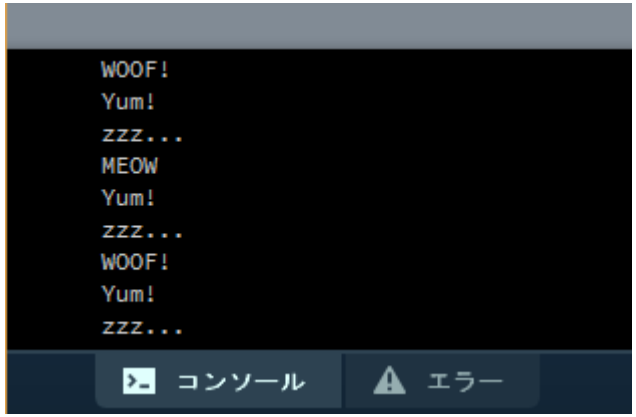
```
class Cat {

  int age;

  Cat(){
    age = 0;
  }

  void eat(){
    println("Yum!");
  }

  void sleep(){
    println("zzz...");
  }

  void meow(){
    println("MEOW");
  }

}
```

フィールド
共通点

メソッド
共通点

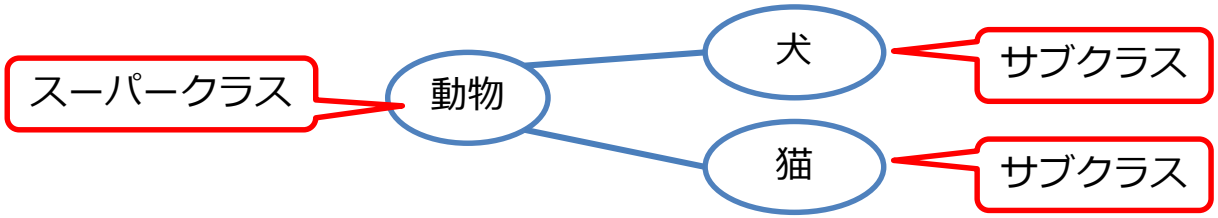
メインタブ

```
Dog dog;              //犬オブジェクト
Cat cat;              //猫オブジェクト

void setup(){
  dog = new Dog();
  cat = new Cat();
}

void draw(){
  dog.eat();                    //犬が食べる
  dog.sleep();                  //犬が寝る
  dog.bark();                   //犬が吠える

  cat.eat();                    //猫が食べる
  cat.sleep();                  //猫が寝る
  cat.meow();                   //猫が鳴く

}
```



「動物」クラス

- 犬・猫クラスの共通点を「動物」クラスとして定義
  - 共通するフィールドやメソッドは、動物クラスで定義、犬・猫クラスは動物クラスのフィールドやメソッドを引き継ぐ
    - スーパークラス（親クラス）：継承元のクラス
    - サブクラス（子クラス）：継承した新しいクラス



Animalクラス

```
class Animal {

  int age;                    //年齢は犬・猫クラス共通なので、動物クラスで定義

  Animal(){                   //コンストラクタ
    age = 0;
  }

  void eat(){                 //食べるは犬・猫クラス共通なので、動物クラスで定義
    println("Yum!");
  }

  void sleep(){               //寝るは犬・猫クラス共通なので、動物クラスで定義
    println("zzz...");
  }

}
```

Animalクラスを継承したDogクラス

```
class Dog extends Animal {

  Dog(){
    super();                    //スーパークラスのコンストラクタを実行
  }

  void bark(){                  //吠えるは犬クラス独自のメソッド
    println("WOOF!");
  }

}
```

スーパークラスAnimalを継承する

共通するフィールドやメソッドは
Animalクラスで定義しているため
Dogクラスから削除

Animalクラスを継承したCatクラス

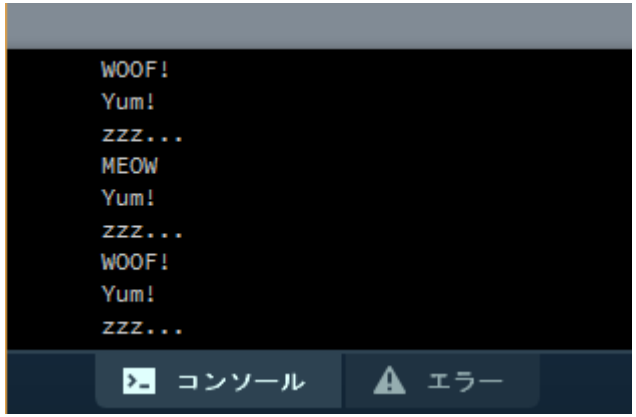
```
class Cat extends Animal {

  Cat(){
    super();                    //スーパークラスのコンストラクタを実行
  }

  void meow(){                  //鳴くは猫クラス独自のメソッド
    println("MEOW");
  }

}
```

スーパークラスAnimalを継承する

共通するフィールドやメソッドは
Animalクラスで定義しているため
Catクラスから削除

メインタブ

```
Dog dog;              //犬オブジェクト
Cat cat;              //猫オブジェクト

void setup(){
  dog = new Dog();
  cat = new Cat();
}

void draw(){
  dog.eat();                    //犬が食べる
  dog.sleep();                  //犬が寝る
  dog.bark();                   //犬が吠える

  cat.eat();                    //猫が食べる
  cat.sleep();                  //猫が寝る
  cat.meow();                   //猫が鳴く

}
```



注意点

- クラスを継承する
  - `class サブクラス名 extends スーパークラス名{}`
- スーパークラスのコンストラクタを呼び出す
  - `super()`
- スーパークラスのメソッドを呼び出す
  - `super.メソッド名()`

課題08　壁・反射板・ブロックの共通点

- 課題07に壁クラスを追加することを想定
  - フィールド
    - x座標、y座標、幅、高さ
  - メソッド
    - ボールと(左・右・上・下)側で衝突したかどうか
    - ボールと頂点で衝突したかどうか
    - 描く

壁・反射板・ブロッククラスの共通点

		反射板	ブロック	壁	共通点
フィールド	x座標、y座標	○	○	○	○
	幅、高さ	○	○	○	○
	速度	○	-	-	
	見えているかどうか	-	○	-	
メソッド	コンストラクタ	-	-	-	
	移動する	○	-	-	
	消える	-	○	-	
	描く	○	○	○	○
	ボールと(左右上下)側で衝突するかどうか	○	○	○	○
	ボールと頂点で衝突するかどうか	○	○	○	○

「四角いもの」クラス

- 共通点を「四角いもの」クラス（クラス名 Rect）として定義
- 反射板・壁・ブロックは「四角いもの」クラスを継承する
  - 課題08では反射板クラスのみ作成

Kadai_08

```
//メインタブ

Bar bar;

void setup() {
  size(400, 400);
  ellipseMode(RADIUS);
  bar = new Bar(width/2, height*0.8);
}

void draw() {
  background(0);

  bar.move();

  bar.display();
}
```

```
//Rectクラス（スーパークラス）

class Rect {
```

フィールドの追加

x座標、y座標、幅、高さ

コンストラクタの作成とフィールドへの代入

x座標、y座標、幅、高さの
4つのフィールドを初期化する

メソッドの追加

描くを定義する（他は今回は省略）

```
}
```

Kadai_08

```
//反射板クラス（サブクラス）

class Bar
```

継承

フィールドの追加
共通点以外のフィールドを宣言する

```
  Bar(float tempX, float tempY){
```

スーパークラスのコンストラクタを使って実行

```
  }
  void move(){
  if (keyPressed && key == CODED) {
      if (keyCode == LEFT) {
        x -= speed;
      } else if (keyCode == RIGHT) {
        x += speed;
      }
    }
    x = constrain(x, 0, width - w);
  }
}
```

```
//Blockクラス，Ballクラスは次回作成する
```